CHAOS DOMINO: REAL-TIME MULTIPLAYER ANDROID APPLICATION WITH MVVM ARCHITECTURE AND FIREBASE

Juan Sebastian Pineda Pineda, Samuel Julian Castillo Muñoz

Technological Faculty, Universidad Distrital Francisco José de Caldas Bogotá

Component-Based Programming - Professor: Jorge Eduardo Hernández R.

Repository: https://github.com/Samuel-uxn/Domino-Caos (DominoCaos2 branch)

ABSTRACT

This document presents the design and implementation of "Chaos Domino", an Android application developed in Kotlin that allows two to four players to play domino matches in virtual rooms synchronized in real-time using Firebase. The game extends traditional dominoes with a two-turn power and random events that temporarily alter the rules, the score, the boneyard, or the turn. The application follows the Model-View-ViewModel (MVVM) pattern, isolates the game logic in an engine independent of Android, and builds the interface with reusable components in Jetpack Compose. The requirements, architecture, class diagram, components, match flow, and synchronization strategy are described.

KEYWORDS: Android, Kotlin, MVVM, Jetpack Compose, Firebase, Firestore, multiplayer game, real-time synchronization, unit testing.

I. INTRODUCTION

The development of multiplayer mobile applications requires a clear separation of business logic from the graphical interface, as well as solving the concurrency problem among multiple clients modifying the same shared state.

The project objectives are:

1. Implement a functional game with user registration and login.
2. Synchronize the match in real-time across all devices using Firebase.
3. Apply the MVVM pattern, separating the game logic from the interface.
4. Build the interface with reusable components.
5. Validate the logic through unit testing.

II. DESIGN SPECIFICATION

A. General game description

Each player receives 7 tiles from a double-six set (28 tiles, from 0-0 to 6-6). The remaining tiles form the boneyard. Players take turns placing a tile whose value matches one of the two ends of the board. If a player has no valid move, they must draw from the boneyard; if the boneyard is empty, they pass their turn. The objective is to obtain the highest score at the end.

B. Functional requirements

- RF-01 User registration and login with email and password.
- RF-02 Room creation with a 4-character code and joining via code.
- RF-03 Support for 2 to 4 players per room.
- RF-04 Distribution of 7 tiles per player and creation of the boneyard.
- RF-05 Control of the current turn throughout the match.
- RF-06 Validation of moves according to domino rules.
- RF-07 Selection and placement of a tile on the left or right end.
- RF-08 Drawing a tile when there is no valid move; loss of turn if the boneyard is empty.
- RF-09 Use and control of powers with a limited number of uses, selected before the move.
- RF-10 Score calculation per play, domino bonus, and tie-breaking.
- RF-11 Generation and application of random events.
- RF-12 Automatic detection of the end of the match (domino, block, maximum rounds, or abandonment).
- RF-13 Abandoning the match and restarting by the room creator.
- RF-14 Permanent display of the turn, the score, and the board state.

C. Non-functional requirements

- RNF-01 Real-time synchronization between devices using Firebase.
- RNF-02 Consistency during simultaneous actions (use of transactions).
- RNF-03 Separation between game logic and graphical interface.
- RNF-04 MVVM Architecture.
- RNF-05 Reusable components for the board, tiles, players, powers, and controls.

- RNF-06 Verifiable domain logic without Android dependencies.
- RNF-07 Compatibility with Android 7.0 (API 24) or higher.

D. Powers

The implemented power is SKIP (SALTO): by placing a tile while activating this power, the next player loses their turn. Each player has 2 uses. In a two-player match, the effect allows the same player to play again. The design, based on an enum (Poder), allows adding new powers (Wildcard, Swap, Reverse) without modifying the general structure.

E. Scoring

- When placing a tile: the sum of its two values, plus 5 points if it is a double.
- With the DOUBLE_POINTS (PUNTOS_DOBLES) event, the value is multiplied by 2; with NO_POINTS (SIN_PUNTOS), it is 0.
- Domino bonus: the player who runs out of tiles receives the sum of the points from the tiles held by their active rivals.
- Winner: highest score. In case of a tie, the winner is the one who used fewer powers; if the tie persists, the match is declared a draw.

F. Random events

At the start of each turn, there is a 25% probability that a randomly chosen event will occur (Table I).

Event	Duration	Effect
DOUBLE_POINTS	2 turns	Placed tiles are worth double
NO_POINTS	2 turns	Placed tiles score zero
FREE_DOUBLES	2 turns	Doubles can be placed on either side
FORCED_DRAW	Immediate	The current player draws a tile
EXTRA_TURN	Immediate	The current player plays twice
DISCARD_TO_BONEYARD	Immediate	Returns a random tile to the boneyard

III. ARCHITECTURE

A. General overview

The application is divided into layers, and each handles a single responsibility. This makes the code easier to understand, test, and modify. There are five layers:

- View: These are the screens. They only show what is happening in the match and notify when the user taps something. They do not know the domino rules.
- ViewModel: This is the intermediary. It receives what the user did, converts it into a game action, and delivers the updated state to the screen.
- Domain: This is the "rulebook" of the game, written as code. It decides if a play is valid, calculates points, manages turns, and triggers events. It does not depend on Android or Firebase.
- Data: Responsible for saving and reading the match in Firebase.
- Model: These are the structures that describe the game (tiles, players, match state). All other layers use them.

The layers communicate in a single direction: the View talks to the ViewModel, the ViewModel talks to the Domain and Data, and the Domain only uses the Model. No lower layer knows about the layers above it.

B. How MVVM is applied

MVVM stands for Model, View, and ViewModel. In this project, it is distributed as follows:

Model. This includes the model, domain, and data packages. The complete state of a match lives in a single immutable object (EstadoPartida): tiles of each player, board, boneyard, turn, scores, and active event. Because it is immutable, each play does not modify the old state but creates a new one. This prevents errors and facilitates testing. The rules engine (MotorPartida) receives the current state and an action, and returns the new state or an error message.

View. These are @Composable functions in Jetpack Compose. They receive the state and draw it; when the user taps something, they notify the ViewModel. To know which buttons to enable, they use OpcionesTurno, ensuring rules are not repeated in the interface.

ViewModel. PartidaViewModel holds what the screen needs to see (room state, errors, selected tile, and chosen power) using StateFlow. When any of these change, Compose redraws the screen automatically. AuthViewModel handles registration and login with Firebase Authentication.

C. What happens when someone plays

The flow is always the same and goes in a single direction:

1. The user taps a tile and a side of the board.
2. The View notifies the ViewModel.
3. The ViewModel creates the action and asks the repository to execute it.
4. The repository opens a transaction in Firestore, reads the current room state, and passes it to the engine.
5. The engine checks that the play is valid and calculates the new state.
6. If it was valid, the repository saves the new state in Firestore. If not, nothing is saved and the error is displayed.
7. Firestore notifies all devices connected to that room.
8. Each device updates its state and Compose redraws the screen.

Importantly, the phone that made the play does not update its screen "on its own": it receives the new state just like the others. This way, everyone sees exactly the same thing.

D. Real-time synchronization

We do not use a custom server. Each room is a document in Cloud Firestore, inside the salas (rooms) collection, and the room code is the document identifier. That document is the shared "table": it saves everything that happens in the match.

Synchronization works with an addSnapshotListener, which is like subscribing to the document: every time someone modifies it, Firestore notifies everyone who is subscribed.

To prevent two players from overwriting each other when acting almost simultaneously, every modification is made inside a transaction. This means it reads, applies the rule, and writes as a single operation. If the state changed in the meantime, Firestore repeats the operation with the most recent state, and the engine rejects the play if it is no longer valid (for example, if it is no longer that player's turn).

The room code is generated with 4 random characters, taken from a 32-symbol alphabet (excluding the letters I and O, and the numbers 0 and 1, to avoid confusion). When creating the room, a transaction checks that one with that code does not already exist; if it does, another is generated, retrying up to five times. When joining, the typed text is cleaned (spaces removed and converted to uppercase) and the document is searched: if it does not exist, a notification is shown.

E. How data is stored in Firestore

Firestore only stores basic types (texts, numbers, lists, and maps), not Kotlin objects. For this reason, EstadoSerializador exists, which converts the match state to a map to save it and converts it back when reading. Tiles are saved as text (e.g., "3-4") and enums by their name. If any field is missing or arrives with another type, a default value is used so the application does not crash.

Each room document saves: the code, the match phase (waiting, playing, or finished), the list of players (with their hand, score, power uses, and if they abandoned), the board, the boneyard, the current turn, the round, the active event, the last message, and, upon finishing, the winner and the reason for ending.

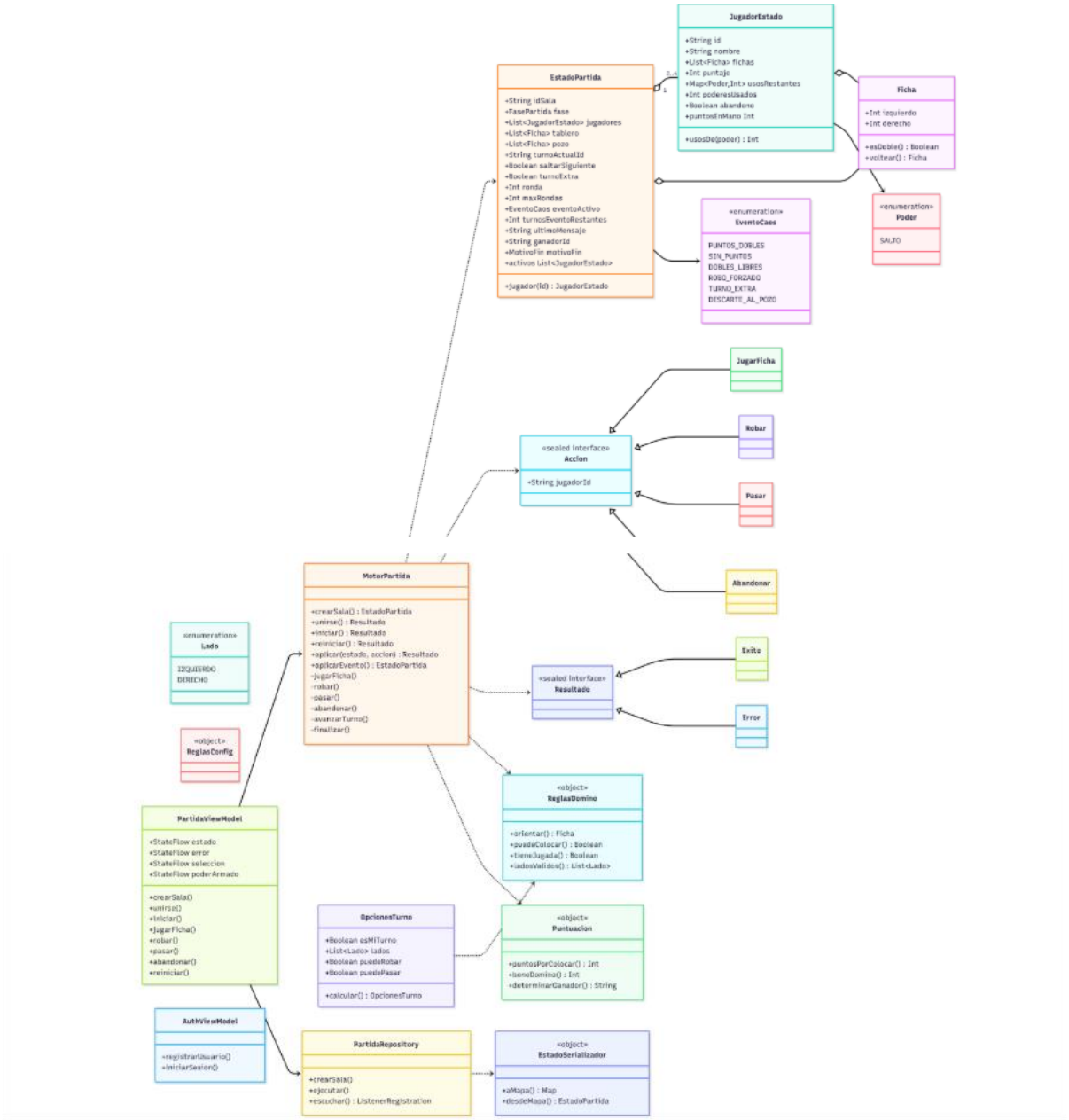
F. Navigation

The application has a single Activity (MainActivity) that decides which screen to show based on the state:

- If there is no active session, it shows Login or Register.
- If there is a session but no room, it shows the main menu.
- If there is a room, it shows a screen based on the match phase: waiting room, game, or end of match.

There is no need to manually "navigate" between match screens: when the creator starts the game or the match ends, the state in Firestore changes and all devices automatically transition to the corresponding screen.

IV. CLASS DIAGRAM



V. INTERFACE COMPONENTS

The interface was built with reusable components (view.componentes package) that receive data as parameters and return events via lambda functions, without knowing the game logic (Table II).

Component	Responsibility
FichaView	Draws a tile with Canvas (dots on a 3x3 grid). If the tile is null, it is shown face down. Supports "selected" state and optional click.

TableroView	Displays the tiles on the table with horizontal scrolling and both ends open.
JugadorInfo	Card with name, score, tiles, and used powers; highlights the player whose turn it is.
PanelPoderes	Power buttons with remaining uses; allows "arming" the power before placing the tile.
Controles	Left, Right, Draw, Pass, and Exit buttons, enabled according to OpcionesTurno.
Estilo	Game Boy palette (Gameboy object), TextoGameboy, and BotonGameboy.

Screens: Login, Register, MenuPrincipal, SalaEspera (Waiting Room), Juego (Game), and FinPartida (End of Match). The Game screen composes the previous components, shows the turn, round, active event, and last message, and requests confirmation before abandoning the match.

VI. EVENT HANDLING AND MATCH FLOW

A. Match phases

The match goes through three phases: ESPERANDO (open room), JUGANDO (playing), and FINALIZADA (finished). Only the room creator (first active player) can start it (minimum 2 players) and restart it after it finishes.

B. Processing a play (MotorPartida.aplicar)

1. It is validated that the match is in progress, that the player belongs to it, has not abandoned, and that it is their turn.
2. PlayTile (JugarFicha): the tile is searched for in the hand, the power is verified (remaining uses), the tile is oriented according to the chosen end, the board is updated, the score is calculated, and the power use is deducted. If the hand is empty, the match ends by DOMINO; otherwise, the turn advances.
3. Draw (Robar): only if there is no valid play and the boneyard is not empty.
4. Pass (Pasar): only if there is no valid play and the boneyard is empty.

5. Abandon (Abandonar): in the waiting room, it removes the player; in-game, it marks them as abandoned and returns their tiles to the boneyard. If fewer than two active players remain, the match ends.

C. Turn advancement (avanzarTurno)

1. The turn counter for the round is incremented; when all active players have completed their turns, the round is incremented. If the maximum number of rounds is exceeded, the match ends (MotivoFin.RONDAS).
2. One turn is deducted from the active temporary event, and it is removed when it expires.
3. The next player is determined: the current one repeats if there is an EXTRA_TURN; otherwise, it advances to the next active player, skipping one more if a SKIP is pending.
4. If the boneyard is empty and no player can act, the match ends by BLOCK.
5. If the player whose turn it is cannot act, it is announced and passes to the next.
6. With a 0.25 probability, a random event is triggered.

D. End of match

The causes are DOMINO, BLOCK, ROUNDS, and ABANDONMENT (MotivoFin). Upon finishing, the domino bonus is applied if applicable, the winner is determined based on score and tie-breakers, events and flags are cleared, and the final message is published. The FinPartida screen shows the winner, the reason, and the score table.

VII. TECHNOLOGIES AND ENVIRONMENT

- Language: Kotlin 2.2.10.
- Interface: Jetpack Compose (BOM 2026.02.01) with Material 3.
- Concurrency and state: Kotlin Coroutines and StateFlow.
- Backend: Firebase Authentication (email and password) and Cloud Firestore.
- Compilation: Android Gradle Plugin 9.3.3; minSdk 24, targetSdk 37.
- Testing: JUnit 4.13.2.
- Version control: Git and GitHub.

VIII. LIMITATIONS AND FUTURE WORK

- Security: the engine runs on the client, and the room document contains the tiles of all players. A technical user could discover rival hands or alter the state. An improvement

is to move validation to Cloud Functions and separate private hands into subcollections with per-player security rules.

- Data cleanup: finished rooms are not deleted from Firestore; a cleanup task or an expiration field is proposed.
- Powers: only Skip was implemented (minimum required); the design allows adding Wildcard, Swap, and Reverse by extending the Poder enum and the engine.
- Authentication: Google Sign-In could be added (optional in the workshop) along with logout functionality.
- Interface: configuring the number of rounds, match history, and user profile, present in the mockups, remain as future work.
- Navigation: replace string-based navigation with a typed navigation library.

IX. CONCLUSIONS

A functional Android application was developed that meets the workshop requirements: real-time synchronized multiplayer matches, powers, random events, scoring, automatic detection of the end of the match, abandonment, and restarting. The MVVM architecture, along with a rules engine independent of Android, facilitated maintenance and allowed verifying the logic with 52 unit tests. The use of Firestore transactions guarantees consistency during simultaneous actions, and reusable components allow extending the interface with minimal code duplication. As future work, it is proposed to reinforce the security of the shared state and expand the catalog of powers.

REFERENCES

[1] Google, "Guide to app architecture," Android Developers. [Online]. Available: https://developer.android.com/topic/architecture

[2] Google, "Jetpack Compose," Android Developers. [Online]. Available: https://developer.android.com/jetpack/compose

[3] Google, "Cloud Firestore documentation," Firebase. [Online]. Available: https://firebase.google.com/docs/firestore

[4] Google, "Firebase Authentication," Firebase. [Online]. Available: https://firebase.google.com/docs/auth

[5] JetBrains, "Kotlin documentation." [Online]. Available: https://kotlinlang.org/docs/home.html

[6] ISO/IEC/IEEE 29148:2018, "Systems and software engineering - Life cycle processes - Requirements engineering," 2018.

[7] Universidad Distrital Francisco José de Caldas, "Programación por Componentes - Taller 02: Dominó Caos," 2026.